ICS 2026 Lab02 实验报告

谈嘉芸 23307090074
GitHub 仓库链接：https://github.com/TANJIAYUN0817/ics-2026-bomblab

实验 1

首先使用 objdump 查看入口函数：

```
objdump -d --no-show-raw-insn --disassemble=phase_01 ./bomb
```

先从入口函数判断口令格式和检查方式。调用 valid_u32_fields 前，程序将 1 放入 esi；随后 sscanf 的返回值也必须等于 1。这说明本关要求输入一个无符号整数。输入被保存到栈上的-0x14(%rbp) 位置，入口函数调用 p1_compute，再将它的返回值与输入整数比较。

关键汇编：

```
# 要求一个整数
40209d: mov    $0x1,%esi
# 输入整数的保存位置
4020c2: lea    -0x14(%rbp),%rdx
# sscanf 必须读到一个整数
4020d3: cmp    $0x1,%eax
# rdi = challenge
4020d8: mov    %r12,%rdi
# 调用 p1_compute，比较其返回值与输入
4020db: call   p1_compute
4020e0: cmp    %eax,-0x14(%rbp)
4020e3: sete   %al
```

继续反汇编 p1_compute：

```
objdump -d --no-show-raw-insn --disassemble=p1_compute ./bomb
```

p1_compute 先从 challenge 中取出不同位段，再依次调用 p1_step1、p1_step0 和 p1_step2。

关键汇编：

```
# eax = challenge >> 32 的低 32 位
40205e: shr    $0x20,%rax
# esi = challenge >> 16 的低 32 位
402062: shr    $0x10,%rsi
# 保存 high，作为 p1_step1 的第一个参数
402066: mov    %eax,%r8d
402069: mov    %eax,%edi
# p1_step1(high, middle)
40206e: call   p1_step1
# p1_step0 的参数为 low、high；保存 p1_step1 的返回值
402073: mov    %r8d,%esi
```

```
402076: mov    %edx,%edi
402078: mov    %eax,%ecx
# p1_step0(low, high)
40207a: call   p1_step0
# p1_step2 的参数为 p1_step0、p1_step1 的返回值
40207f: mov    %ecx,%esi
402081: mov    %eax,%edi
# p1_step2(step0 结果, step1 结果)
402083: call   p1_step2
```

调用关系：

s1 = p1_step1(high, middle)
s0 = p1_step0(low, high)
ans = p1_step2(s0, s1)

反汇编辅助函数并恢复计算关系

```
objdump -d --no-show-raw-insn --disassemble=p1_step0 ./bomb
objdump -d --no-show-raw-insn --disassemble=p1_step1 ./bomb
objdump -d --no-show-raw-insn --disassemble=p1_step2 ./bomb
```

p1_step1 先用 sub 计算两个参数之差，再用 lea 完成乘 3 和加常数，所以 t1 = 3 × (high - middle) + 0xfcf6

关键汇编：

```
# edi = edi - esi
402034: sub    %esi,%edi
# eax = edi + 2*edi + 0xfcf6
402036: lea    0xfcf6(%rdi,%rdi,2),%eax
```

p1_step0 用 lea 计算第一个参数加第二个参数的两倍，再与常数异或，所以 t0 = (low + 2 × high) ^ 0x3085

关键汇编：

```
# eax = edi + 2*esi
402024: lea    (%rdi,%rsi,2),%eax
# eax ^= 0x3085
402027: xor    $0x3085,%eax
```

p1_step2 先用 lea 计算第一个参数的五倍，再加上第二个参数并异或常数，所以最终答案为 answer = (5 × t0 + t1) ^ 0x7733

关键汇编：

```
# eax = edi + 4*edi = 5*edi
402044: lea    (%rdi,%rdi,4),%eax
# eax += esi
402047: add    %esi,%eax
# eax ^= 0x7733
402049: xor    $0x7733,%eax
```

最后将 answer 存入 values[0]，并令 count = 1，由模板提供的 write_answer 输出一个无符号十进制整数。

实验 2

使用 objdump 查看入口函数：

```
objdump -d --no-show-raw-insn --disassemble=phase_02 ./bomb
```

先从入口函数判断输入格式和检查方式。调用 valid_u32_fields 前，程序将 6 放入 esi，说明本关要求输入 6 个无符号整数。随后 sscanf 的返回值也必须为 6，确认程序成功读到了 6 个数。输入被依次保存到栈上从 -0x40(%rbp) 开始的位置。

关键汇编：

```
# 要求输入 6 个无符号整数
4020fd: mov    $0x6,%esi
40210c: call   valid_u32_fields
# 为 sscanf 准备格式串和各整数的保存位置
40212a: lea    -0x2c(%rbp),%rax
40212e: lea    -0x3c(%rbp),%rcx
402132: mov    $0x40317a,%esi
402137: push   %rax
402138: lea    -0x30(%rbp),%rax
40213c: lea    -0x40(%rbp),%rdx
402140: mov    %r12,%rdi
402143: push   %rax
402144: lea    -0x34(%rbp),%r9
402148: lea    -0x38(%rbp),%r8
40214c: xor    %eax,%eax
40214e: call   __isoc99_sscanf@plt
# sscanf 必须成功读取 6 个整数
402155: cmp    $0x6,%eax
402158: jne    ...
```

确认输入个数后，继续查看输入数之间的关系。入口函数先将 challenge 右移 4 位，再取低 4 位，并与第一个输入数比较，由此得到：values[0] = (uint32_t)((challenge >> 4) & 0xfu);

关键汇编：

```
# 取出 challenge 右移 4 位后的低 4 位
40215a: mov    %r13,%rax
402160: shr    $0x4,%rax
402164: and    $0xf,%eax
# 与第一个输入数比较
40215d: mov    -0x40(%rbp),%esi
402167: cmp    %eax,%esi
402169: jne    ...
```

研究循环中的递推关系，每次循环 esi 保存前一个输入数：第一次循环时是 values[0]；之后每轮会更新为刚读取的当前输入数。

关键汇编：

```
# 循环索引 i 从 1 开始
40216b: mov    $0x1,%edx
# ecx = i + 4，作为右移量
402170: lea    0x4(%rdx),%ecx
# rax = challenge
402173: mov    %r13,%rax
# rax = challenge >> (i + 4)
402176: shr    %cl,%rax
# ecx = 2 * i
402179: lea    (%rdx,%rdx,1),%ecx
# eax = (challenge >> (i + 4)) & 0xf
40217c: and    $0xf,%eax
# eax = values[i-1] + 4 位片段
40217f: add    %esi,%eax
# esi = values[i]（当前输入数）
402181: mov    -0x40(%rbp,%rdx,4),%esi
# eax = (values[i-1] + 4 位片段) ^ (2 * i)
402185: xor    %ecx,%eax
# 当前输入数必须等于 eax
402187: cmp    %eax,%esi
402189: jne    ...
# i++，检查完 i = 1 到 5 后退出
40218b: add    $0x1,%rdx
40218f: cmp    $0x6,%rdx
402193: jne    402170 <phase_02+0x80>
```

因此，对于 i = 1 到 5，当前输入数必须满足 values[i] = (values[i - 1] + ((challenge >> (i + 4)) & 0xf)) ^ (2 × i)，将第一个输入数和循环中的递推关系合并，& 0xf 保留 4 位，所以每轮从 challenge 中取出一个 0 到 15 的数；^ 表示按位异或。

values 中共需填写 6 个整数，因此令 count = 6，再由 write_answer 将它们按空格分隔写入输出。

实验 3

首先使用 objdump 查看入口函数：

```
objdump -d --no-show-raw-insn --disassemble=phase_03 ./bomb
```

先从入口函数判断口令格式和检查方式。调用 valid_u32_fields 前，程序将 2 放入 esi，随后 sscanf 的返回值也必须等于 2，说明本关要求输入两个无符号整数。第一个数被保存到 -0x18(%rbp)，第二个数被保存到 -0x14(%rbp)。

入口函数先计算 a = challenge & 7，要求第一个输入等于 a；再计算 b = (challenge >> 32) ^ (challenge >> 3)，调用 p3_dispatch(a, b)，要求第二个输入等于其返回值。

关键汇编：

```
# 要求两个整数
4022ad: mov    $0x2,%esi
# 第一个数存入 rbp-0x18，第二个数存入 rbp-0x14
4022d2: lea    -0x14(%rbp),%rcx
4022d6: lea    -0x18(%rbp),%rdx
# sscanf 必须读到两个整数
4022e7: cmp    $0x2,%eax
# edi = challenge & 7
4022ec: mov    %r12d,%edi
4022f1: and    $0x7,%edi
# 第一个输入必须等于 challenge & 7
4022f4: cmp    %edi,-0x18(%rbp)
# esi = (challenge >> 32) ^ (challenge >> 3)
4022f9: mov    %r12,%rax
4022fc: mov    %r12,%rsi
4022ff: shr    $0x3,%rax
402303: shr    $0x20,%rsi
402307: xor    %eax,%esi
# 调用 p3_dispatch(a, b)，比较其返回值与第二个输入
402309: call   p3_dispatch
40230e: cmp    %eax,-0x14(%rbp)
402311: sete   %al
```

继续反汇编 p3_dispatch：

```
objdump -d --no-show-raw-insn --disassemble=p3_dispatch ./bomb
```

p3_dispatch 是一个 switch 跳转表。它先把第一个参数 a 存入 eax，把第二个参数 b 搬进 edi，然后比较 a 与 7，用 a 查表 0x405820 跳到对应的 p3_case_N，等进入 case 后，edi 里是 b。

关键汇编：

```
# eax = a，edi = b
402244: mov    %edi,%eax
402246: mov    %esi,%edi
# a > 7 则返回 0
402248: cmp    $0x7,%eax
40224b: ja     402295
# 按 a 查跳转表
40224d: notrack jmp *0x405820(,%rax,8)
...
402295: xor    %eax,%eax
402297: ret
```

跳转表项不是按 a 顺序排列的，必须用 GDB 读真实表内容

通过 GDB 确定跳转表的顺序：

```
gdb -batch -q -ex 'x/8gx 0x405820' ./bomb
```

a	表项地址	跳到
0	0x402268	p3_case_1
1	0x402270	p3_case_4
2	0x402278	p3_case_0
3	0x402280	p3_case_3
4	0x402288	p3_case_2
5	0x402290	p3_case_5
6	0x402258	p3_case_7
7	0x402260	p3_case_6

反汇编 8 个 case

```
for i in 0 1 2 3 4 5 6 7; do
  objdump -d --no-show-raw-insn --disassemble=p3_case_$i ./bomb
done
```

分开观察判断具体表达式：

p3_case_0：进入时 edi = b。lea 0xfa93(%rdi,%rdi,4),%eax 计算 eax = b + 4b + 0xfa93 = 5b + 0xfa93，所以 p3_case_0(b) = 5 × b + 0xfa93。

关键汇编：

```
# eax = b + 4*b + 0xfa93
4021a4: lea    0xfa93(%rdi,%rdi,4),%eax
```

p3_case_1：先用 lea (%rdi,%rdi,1),%eax 计算 eax = b + b = 2b，再 xor $0xa53c 与常数异或，所以 p3_case_1(b) = (2 × b) ^ 0xa53c。

关键汇编：

```
# eax = b + b = 2*b
4021b4: lea    (%rdi,%rdi,1),%eax
# eax ^= 0xa53c
4021b7: xor    $0xa53c,%eax
```

p3_case_2：lea 0x372fd(%rdi,%rdi,4),%eax 计算 eax = b + 4b + 0x372fd = 5b + 0x372fd，所以 p3_case_2(b) = 5 × b + 0x372fd。

关键汇编：

```
# eax = b + 4*b + 0x372fd
4021c4: lea    0x372fd(%rdi,%rdi,4),%eax
```

p3_case_3：先把 b 复制到 eax，再把 edi 右移 2 位得到 b >> 2，然后 eax ^= 0xf807 得 b ^ 0xf807，最后加 b >> 2，所以 p3_case_3(b) = (b ^ 0xf807) + (b >> 2)。

关键汇编：

```
# eax = b
4021d4: mov    %edi,%eax
# edi = b >> 2
4021d6: shr    $0x2,%edi
# eax = b ^ 0xf807
4021d9: xor    $0xf807,%eax
# eax = (b ^ 0xf807) + (b >> 2)
4021de: add    %edi,%eax
```

p3_case_4：lea 0xe8ab(%rdi,%rdi,4),%eax 计算 eax = b + 4b + 0xe8ab = 5b + 0xe8ab，所以 p3_case_4(b) = 5 × b + 0xe8ab。

关键汇编：

```
# eax = b + 4*b + 0xe8ab
4021f4: lea    0xe8ab(%rdi,%rdi,4),%eax
```

p3_case_5：lea 0x0(,%rdi,4),%eax 计算 eax = 4b，再 xor $0x9ba8 与常数异或，所以 p3_case_5(b) = (4 × b) ^ 0x9ba8。

关键汇编：

```
# eax = 4*b
402204: lea    0x0(,%rdi,4),%eax
# eax ^= 0x9ba8
40220b: xor    $0x9ba8,%eax
```

p3_case_6：先把 b 复制到 eax，再把 edi 右移 1 位得到 b >> 1，然后 eax ^= 0xb154 得 b ^ 0xb154，最后加 b >> 1，所以 p3_case_6(b) = (b ^ 0xb154) + (b >> 1)。

关键汇编：

```
# eax = b
402224: mov    %edi,%eax
# edi = b >> 1
402226: shr    $1,%edi
# eax = b ^ 0xb154
402228: xor    $0xb154,%eax
# eax = (b ^ 0xb154) + (b >> 1)
40222d: add    %edi,%eax
```

p3_case_7：lea 0x40970(%rdi,%rdi,4),%eax 计算 eax = b + 4b + 0x40970 = 5b + 0x40970，所以 p3_case_7(b) = 5 × b + 0x40970。

关键汇编：

```
# eax = b + 4*b + 0x40970
402234: lea    0x40970(%rdi,%rdi,4),%eax
```

结合跳转表可得：

a = 0: values[1] = (2 × b) ^ 0xa53c	（p3_case_1）
a = 1: values[1] = 5 × b + 0xe8ab	（p3_case_4）
a = 2: values[1] = 5 × b + 0xfa93	（p3_case_0）
a = 3: values[1] = (b ^ 0xf807) + (b >> 2)	（p3_case_3）
a = 4: values[1] = 5 × b + 0x372fd	（p3_case_2）
a = 5: values[1] = (4 × b) ^ 0x9ba8	（p3_case_5）
a = 6: values[1] = 5 × b + 0x40970	（p3_case_7）
a = 7: values[1] = (b ^ 0xb154) + (b >> 1)	（p3_case_6）

最后将 values[0]、values[1] 存入数组，count = 2，由 write_answer 输出两个无符号十进制整数。

实验 4

使用 objdump 查看入口函数：

```
objdump -d --no-show-raw-insn --disassemble=phase_04 ./bomb
```

先从入口函数判断输入格式和检查方式。调用 valid_u32_fields 前，程序将 1 放入 esi，说明本关要求输入 1 个无符号整数。随后 sscanf 的返回值也必须为 1，确认程序成功读到了 1 个数。

关键汇编：

```
# 要求输入 1 个无符号整数
```

```
40239d: mov    $0x1,%esi
4023aa: call   valid_u32_fields
# 为 sscanf 准备输入字符串和整数的保存位置
4023c2: lea    -0x14(%rbp),%rdx
4023c6: mov    $0x403189,%esi
4023cb: mov    %rbx,%rdi
4023ce: call   __isoc99_sscanf@plt
# sscanf 必须成功读取 1 个整数
4023d3: cmp    $0x1,%eax
4023d6: jne    ...
```

确认输入个数后，继续查看 p4_walk 的四个参数。入口函数从 challenge 中取出四个参数，分别放入 rdi、rsi、rdx、rcx，然后调用 p4_walk。

关键汇编：

```
# rcx = challenge
4023d8: mov    %r12,%rcx
# rax = challenge
4023db: mov    %r12,%rax
# edi = challenge 低 32 位
4023de: mov    %r12d,%edi
# esi = 0
4023e1: xor    %esi,%esi
# rax = challenge >> 32
4023e3: shr    $0x20,%rax
# rcx = challenge >> 7
4023e7: shr    $0x7,%rcx
# edi = challenge & 0x7f
4023eb: and    $0x7f,%edi
# edx = 0x7f
4023ee: mov    $0x7f,%edx
# ecx = (challenge >> 7) ^ (challenge >> 32)
4023f3: xor    %eax,%ecx
# 调用 p4_walk(a, b, c, d)
4023f5: call   p4_walk
# 输入整数必须等于 p4_walk 的返回值
4023fa: cmp    %eax,-0x14(%rbp)
4023fd: sete   %al
```

由此可得四个参数为：

```
a = (uint32_t)challenge & 0x7fu;
b = 0;
c = 0x7fu;
d = (uint32_t)(challenge >> 7) ^ (uint32_t)(challenge >> 32);
```

反汇编 p4_walk：

```
objdump -d --no-show-raw-insn --disassemble=p4_walk ./bomb
```

p4_walk 是一个二分查找式递归。四个参数 a、b、c、d 分别放在 edi、esi、edx、ecx。它先计算中点 mid = b + ((c - b) >> 1)，若 mid == a 则进入基准情况返回；否则递归左半或右半，返回时先乘 3，再异或 salt = mid + d，最后左半异或 0xa、右半异或 0x2。

关键汇编：

```
# eax = a
402324: mov    %edi,%eax
# edi = c
402326: mov    %edx,%edi
# edi = c - b
402328: sub    %esi,%edi
# edi = (c - b) >> 1
40232a: shr    $1,%edi
# edi = b + ((c - b) >> 1) = mid
40232c: add    %esi,%edi
# 比较 mid 和 a，若 mid==a 则为基准情况
40232e: cmp    %eax,%edi
402330: je     402380 <p4_walk+0x60>
# eax = mid + 0x70
402380: lea    0x70(%rdi),%eax
# 所以基准情况返回 eax = (mid + 0x70) ^ d
402383: xor    %ecx,%eax
402385: ret
```

在非基准情况下，先保存 salt = mid + d，再比较 a 和 mid：

关键汇编：

```
# ebx = mid + d = salt
402337: lea    (%rdi,%rcx,1),%ebx
40233a: sub    $0x8,%rsp
# 比较 a 和 mid
40233e: cmp    %edi,%eax
```

如果 a > mid，不跳，继续执行 402342，在 [mid+1, c] 里递归：

关键汇编：

```
# esi = mid + 1，新的 b
402342: lea    0x1(%rdi),%esi
# edi = a
402345: mov    %eax,%edi
# 递归 p4_walk(a, mid + 1, c, d)
402347: call   p4_walk
# eax = 3 * 返回值
40234c: lea    (%rax,%rax,2),%eax
# eax ^= salt
40234f: xor    %ebx,%eax
402351: mov    -0x8(%rbp),%rbx
```

```
402355: leave
# eax ^= 0x2
402356: xor    $0x2,%eax
402359: ret
```

右半返回 (3 × p4_walk(a, mid + 1, c, d)) ^ salt ^ 0x2。

如果 a < mid（跳转），执行 402360，在 [b, mid-1] 里递归：

关键汇编：

```
# edx = mid - 1，新的 c
402360: lea    -0x1(%rdi),%edx
# edi = a
402363: mov    %eax,%edi
# 递归 p4_walk(a, b, mid - 1, d)
402365: call   p4_walk
# eax = 3 * 返回值
40236a: lea    (%rax,%rax,2),%eax
# eax ^= salt
40236d: xor    %ebx,%eax
40236f: mov    -0x8(%rbp),%rbx
402373: leave
# eax ^= 0xa
402374: xor    $0xa,%eax
402377: ret
```

左半返回 (3 × p4_walk(a, b, mid - 1, d)) ^ salt ^ 0xa。

最后得出结论：

如果 mid == a 返回 (mid + 0x70) ^ d

如不然，令 salt = mid + d：

如果 a < mid：返回 (3 × p4_walk(a, b, mid - 1, d)) ^ salt ^ 0xa

否则返回 (3 × p4_walk(a, mid + 1, c, d)) ^ salt ^ 0x2

最后将 p4_walk 的返回值存入 values[0]，count = 1，由 write_answer 输出一个无符号十进制整数。

实验 5

首先使用 objdump 查看入口函数：

```
objdump -d --no-show-raw-insn --disassemble=phase_05 ./bomb
```

先从入口函数判断输入格式和检查方式。

调用 valid_u32_fields 前，程序将 2 放入 esi，说明本关要求输入两个无符号整数。随后 sscanf 的返回值也必须为 2，确认程序成功读到了两个数。输入分别保存到 -0x1c(%rbp) 和 -0x18(%rbp)。

之后，程序调用 p5_traverse，并将返回值与第一个输入数比较；p5_traverse 还会通过 rsi 指向的位置写入一个值，程序再将该值与第二个输入数比较。

关键汇编：

```
# 要求输入两个无符号整数
40247d: mov    $0x2,%esi
40248a: call   valid_u32_fields
# 为 sscanf 准备两个整数的保存位置
4024a2: lea    -0x18(%rbp),%rcx
4024a6: lea    -0x1c(%rbp),%rdx
4024aa: mov    $0x403186,%esi
4024af: mov    %rbx,%rdi
4024b2: call   __isoc99_sscanf@plt
# sscanf 必须成功读取两个整数
4024b7: cmp    $0x2,%eax
4024ba: jne    ...
# rsi 指向输出位置，rdi 接收 challenge
4024bc: lea    -0x14(%rbp),%rsi
4024c0: mov    %r12,%rdi
# 调用 p5_traverse(challenge, &x)
4024c3: call   p5_traverse
# 返回值与第一个输入比较
4024c8: mov    %eax,%edx
4024cc: cmp    %edx,-0x1c(%rbp)
4024cf: jne    ...
# 输出位置中的值与第二个输入比较
4024d1: mov    -0x14(%rbp),%eax
4024d4: cmp    %eax,-0x18(%rbp)
4024d7: sete   %al
```

因此，本关的口令由两个整数构成：第一个是 p5_traverse 的返回值，第二个是它通过输出指针写入的值。

继续反汇编 p5_traverse：

```
objdump -d --no-show-raw-insn --disassemble=p5_traverse ./bomb
```

p5_traverse 接收两个参数：rdi 中是 challenge，rsi 中是输出指针。函数先用 challenge 的低 3 位选择头节点，再根据 challenge 的其他位计算初始值和后续遍历步数。

关键汇编：

```
# 保存输出指针
40241a: mov    %rsi,%r8
# 用 challenge 的低 3 位选择头指针表中的一项
40241d: and    $0x7,%eax
402424: mov    0x405b60(,%rax,8),%rdx
# 计算后续遍历步数所需的 challenge >> 3
402420: shr    $0x3,%rcx
```

```
# 计算初始值的一部分
40242c: mov    %rdi,%rax
40242f: shr    $0x20,%rdi
402433: shr    $0x6,%rax
402437: xor    %edi,%eax
40243e: xor    $0x930c,%eax
# 读取头节点的 f4 和 f0，并更新累加值
402439: mov    0x4(%rdx),%esi
402443: add    (%rdx),%eax
402445: lea    (%rax,%rax,4),%eax
402448: mov    %esi,(%r8)
40244b: xor    %esi,%eax
```

节点的 f0 加到累加值后，累加值乘以 5，再与节点的 f4 异或；同时，节点的 f4 被写入输出指针。头节点处理完后，程序通过 (challenge >> 3) & 3 得到额外遍历步数，并沿节点的 next 指针前进。

关键汇编：

```
# 计算额外遍历步数
40243c: mov    %ecx,%edi
40244d: and    $0x3,%edi
# 步数为 0 时，跳过后续循环
402450: je     40246f <p5_traverse+0x5f>
# 循环计数从 1 开始；沿 next 指针移动并递增计数
402452: mov    $0x1,%ecx
402457: mov    0x8(%rdx),%rdx
40245b: add    $0x1,%ecx
# 处理当前节点：累加 f0、乘以 5、写出 f4 并异或
40245e: mov    0x4(%rdx),%esi
402461: add    (%rdx),%eax
402463: lea    (%rax,%rax,4),%eax
402466: mov    %esi,(%r8)
402469: xor    %esi,%eax
# 额外步数尚未走完时继续循环
40246b: cmp    %ecx,%edi
40246d: jae    402457 <p5_traverse+0x47>
```

由此可知，程序先处理头节点，再额外沿 next 前进 (challenge >> 3) & 3 次，因此总共处理 1 到 4 个节点。每到一个节点，按相同关系更新累加值，并将当前节点的 f4 写到输出位置。遍历完成后，eax 是函数返回值，输出位置保存最后一个节点的 f4。

初始累加值和每个节点的更新关系为：

eax = ((challenge >> 6) ^ (challenge >> 32) ^ 0x930c)
eax = ((eax + node->f0) * 5) ^ node->f4

后续节点继续重复第二行运算。

使用 GDB 查看 8 个头指针，以及 12 个节点的数据：

```
gdb -batch -q -ex 'x/8gx 0x405b60' ./bomb
gdb -batch -q -ex 'x/48wx 0x405ba0' ./bomb
```

头指针表中的地址对应节点数组下标：p5_heads = {2, 11, 10, 5, 6, 7, 8, 4}

每个节点占 16 字节：偏移 0 是 f0，偏移 4 是 f4，偏移 8 是 64 位 next 指针。将 next 指针转换为节点数组下标，空指针在求解代码中记为 -1。

节点数据：

下标	f0	f4	next 下标
0	7257	2097	-1
1	19350	1666	9
2	56570	117	11
3	27691	2925	1
4	31477	1864	3
5	55738	364	6
6	33807	2597	7
7	17642	620	8
8	24670	2378	4
9	49711	883	0
10	49635	1129	5
11	52868	1412	10

最后将 p5_traverse 的返回值存入 values[0]，将输出指针写入的最后一个节点 f4 存入 values[1]，设置 count = 2，由 write_answer 输出两个无符号十进制整数。

实验 6

首先使用 objdump 查看入口函数：

```
objdump -d --no-show-raw-insn --disassemble=phase_06 ./bomb
```

先从入口函数判断输入格式和检查方式。

调用 valid_u32_fields 前，程序将 1 放入 esi，说明本关要求输入一个无符号整数。sscanf 的返回值也必须为 1。程序随后调用 p6_dispatch(challenge)，并将它的返回值与输入整数比较。因此需要继续分析 p6_dispatch。

关键汇编：

```
# 要求输入一个无符号整数
4025bd: mov    $0x1,%esi
```

```
4025ca: call   valid_u32_fields
# 将输入解析到栈上，并检查是否成功读到一个整数
4025e2: lea    -0x14(%rbp),%rdx
4025e6: mov    $0x403189,%esi
4025eb: mov    %rbx,%rdi
4025ee: call   __isoc99_sscanf@plt
4025f3: cmp    $0x1,%eax
4025f6: jne    ...
# 调用 p6_dispatch(challenge)，并将返回值与输入比较
4025f8: mov    %r12,%rdi
4025fb: call   p6_dispatch
402600: cmp    %eax,-0x14(%rbp)
402603: sete   %al
```

继续反汇编 p6_dispatch：

```
objdump -d --no-show-raw-insn --disassemble=p6_dispatch ./bomb
```

p6_dispatch 使用位于 0x405e00 的函数指针表 p6_ops，进行两次间接调用。第一次调用前，程序从 challenge 取出低 3 位作为函数表索引，并计算调用参数。第一次调用返回后，程序将返回值与 0xb4b0 异或，作为第二次调用的参数。第二次调用的索引由 challenge 的第 3 到第 5 位决定，最后再加上 0x6294 返回。

关键汇编：

```
# 保存 challenge，并准备第一次调用的参数
402569: mov    %rdi,%rbx
40256c: shr    $0x6,%rdi
402570: mov    %rbx,%rax
402573: shr    $0x20,%rax
402577: xor    %eax,%edi
```

第一次调用的参数为 uint32_t b = (uint32_t)(challenge) >> 6 ^ (uint32_t)(challenge >> 32);

```
# 第一次调用的索引为 challenge 的低 3 位
402579: mov    %rbx,%rax
402583: and    $0x7,%eax
# 根据索引间接调用函数表中的函数
402586: call   *0x405e00(,%rax,8)
# 第一次返回值与 0xb4b0 异或，作为第二次调用参数
40258d: xor    $0xb4b0,%eax
402592: mov    %eax,%edi
# 第二次调用的函数表偏移由 challenge & 0x38 决定
402580: and    $0x38,%ebx
402594: call   *0x405e00(%rbx)
```

表项每个占 8 字节，因此字节偏移 challenge & 0x38 对应的表索引为 uint32_t i2 = ((uint32_t)challenge >> 3) & 7u;

```
# 第二次调用结果加上 0x6294 后返回
40259f: add    $0x6294,%eax
4025a4: ret
```

使用 GDB 查看函数指针表：

```
gdb -batch -q -ex 'x/8gx 0x405e00' ./bomb
```

读出的表项顺序为：

表索引	函数地址	操作函数
0	0x4024e0	p6_op_0
1	0x4024e0	p6_op_0
2	0x402500	p6_op_2
3	0x402520	p6_op_3
4	0x4024f0	p6_op_1
5	0x4024e0	p6_op_0
6	0x402550	p6_op_5
7	0x402540	p6_op_4

表中部分索引指向同一个函数，所以在求解器中用 switch 将表索引映射到相应操作。接着反汇编表中各个不同的函数地址，逐一恢复计算关系：

```
objdump -d --no-show-raw-insn \
  --start-address=0x4024e0 --stop-address=0x402560 ./bomb
```

p6_op_0 将输入与 0x39f9 异或，再加上输入的 8 倍， p6_op_0(x) = (x ^ 0x39f9) + 8*x。

```
4024e4: mov    %edi,%eax
4024e6: xor    $0x39f9,%eax
4024eb: lea    (%rax,%rdi,8),%eax
```

p6_op_1 将 x 循环左移 5 位，再与常数异或， p6_op_1(x) = rol32(x, 5) ^ 0x1dab。

```
4024f4: mov    %edi,%eax
4024f6: rol    $0x5,%eax
4024f9: xor    $0x1dab,%eax
```

p6_op_2 分别计算 4*x + 0x2fb8 和 8*x - x，再将两部分异或， p6_op_2(x) = (4*x + 0x2fb8) ^ (7*x)。

```
402504: lea    0x0(,%rdi,8),%edx
40250b: lea    0x2fb8(,%rdi,4),%eax
402512: sub    %edi,%edx
402514: xor    %edx,%eax
```

p6_op_3 将 x 右移 7 位后与原值异或，再加上常数，p6_op_3(x) = ((x >> 7) ^ x) + 0xd93b。

```
402524: mov    %edi,%eax
402526: shr    $0x7,%eax
402529: xor    %edi,%eax
40252b: add    $0xd93b,%eax
```

p6_op_4 将 x 循环右移 2 位， p6_op_4(x) = ror32(x, 2)。

```
402544: mov    %edi,%eax
402546: ror    $0x2,%eax
```

p6_op_5 将 x 加上常数，再与右移 5 位后的 x 异或，p6_op_5(x) = (x + 0x8427) ^ (x >> 5)。

```
402554: lea    0x8427(%rdi),%eax
40255a: shr    $0x5,%edi
40255d: xor    %edi,%eax
```

其中 rol32 和 ror32 分别表示对 32 位数循环左移和循环右移，移出的位会从另一端补回。

在 solve_phase_6 中，将 p6_dispatch(challenge) 的结果存入 values[0]，设置 count = 1，最后由 write_answer 输出一个无符号十进制整数。

Honor

首先反汇编 secret_gate，分析输入长度与入口格式：

```
objdump -d --no-show-raw-insn --disassemble=secret_gate ./bomb
```

secret_gate 接收输入字符串、challenge 和 context。它先计算输入字符串长度减去 0x24（36），再用无符号条件跳转要求结果不大于 0x7f（127）。因此，输入字符串长度必须在 36 到 163 字节之间。

关键汇编：

```
# r12 指向栈上的输入缓冲区，rdi 接收输入字符串
40272b: lea    -0xd0(%rbp),%r12
...
402749: mov    %rbx,%rdi
40274c: call   strlen@plt
# strlen - 36 必须小于等于 127
402751: sub    $0x24,%rax
402755: cmp    $0x7f,%rax
402759: jbe    402770 <secret_gate+0x60>
```

这里的 jbe 是无符号条件跳转。因此，长度小于 36 时，减法结果会按无符号数回绕成较大的值，无法通过检查；长度上限为 (36+127=163)。

分析 secret_copy 与 “OPEN” 头部检查：

输入长度检查通过后，程序调用 secret_copy，把输入字符串复制到栈上缓冲区。secret_copy 按字节复制，并在复制到字符串结尾的 \0 后停止。

关键汇编：

```
# 将输入字符串复制到 rbp-0xd0 开始的缓冲区
402770: mov    %rbx,%rsi
402773: mov    %r12,%rdi
402776: call   secret_copy
# 逐字节复制，遇到字符串结束符后停止
402700: movzbl (%rsi,%rax,1),%edx
402704: mov    %dl,(%rdi,%rax,1)
402707: add    $0x1,%rax
40270b: test   %dl,%dl
40270d: jne    402700 <secret_copy+0x10>
```

复制完成后，程序检查 rbp-0xb0 处的 4 个字节：

关键汇编：

```
# 检查偏移 0x20 处是否为立即数 0x4e45504f
40277b: cmpl   $0x4e45504f,-0xb0(%rbp)
402785: jne    ...
```

缓冲区起点是 rbp-0xd0，比较位置是 rbp-0xb0，两者相差 0x20，也就是 32 字节。由于 x86-64 使用小端序，立即数 0x4e45504f 在内存中的字节顺序是 4f 50 45 4e，对应 ASCII 字符串 OPEN。因此输入必须先有 32 个字符，再接 OPEN。

检查通过后，程序把 rbp-0xac 处的地址交给 secret_phase。该位置是缓冲区偏移 0x24（36），即 OPEN 后面的内容。

关键汇编：

```
# secret_phase 接收 OPEN 之后的字符串，以及 challenge 和 context
402787: mov    %r14,%rdx
40278a: mov    %r13,%rsi
40278d: lea    -0xac(%rbp),%rdi
402794: call   secret_phase
```

分析 secret_phase 的整数解析与检查：

```
objdump -d --no-show-raw-insn --disassemble=secret_phase ./bomb
```

secret_phase 要求解析 3 个无符号整数，分别保存在栈上的 -0x2c(%rbp)、-0x28(%rbp) 和 -0x24(%rbp)。以下用 v0、v1、v2 表示这三个数。

关键汇编：

```
# 要求解析三个无符号整数
402642: mov    $0x3,%esi
40264f: call   valid_u32_fields
# 为三个整数准备保存位置
402672: lea    -0x24(%rbp),%r8
402676: lea    -0x28(%rbp),%rcx
40267f: lea    -0x2c(%rbp),%rdx
402686: call   __isoc99_sscanf@plt
40268b: cmp    $0x3,%eax
40268e: jne    ...
```

程序先令 ecx = context & 3，并要求 v0 等于该值：

关键汇编：

```
# ecx = context & 3，并与 v0 比较
402690: mov    %r13d,%ecx
402695: and    $0x3,%ecx
402698: cmp    %ecx,-0x2c(%rbp)
40269b: jne    ...
```

所以 v0 = (uint32_t)context & 3u;

根据寄存器运算还原 v1 和 v2：

关键汇编：

```
# 计算 t = ch_high^ ch_low
40269d: mov    %r12,%rsi
4026aa: shr    $0x20,%rsi
4026b2: xor    %r12d,%esi
# 计算 u = cx_high^ cx_low
4026a7: mov    %r13,%r8
4026ae: shr    $0x20,%r8
4026b5: xor    %r13d,%r8d
# 计算 r9 = (v0 * 0x6f35)^ cx_middle
4026a0: imul   $0x6f35,%ecx,%r9d
4026b8: shr    $0x10,%r13
4026be: xor    %r13d,%r9d
```

之后程序将 t^ u 传给 secret_mix，并把返回值与 r9 异或，得到 v1

关键汇编：

```
# secret_mix 的参数为 t^ u
4026bc: mov    %esi,%edi
4026c1: xor    %r8d,%edi
4026c4: call   secret_mix
# v1 = secret_mix(t^ u)^ r9
4026c9: xor    %r9d,%eax
4026cc: cmp    %eax,-0x28(%rbp)
```

```
#secret_mix 对 32 位输入循环左移 5 位，加上 0x6f35，再与 0x8506 异或
402616: rol    $0x5,%eax
402619: add    $0x6f35,%eax
40261e: xor    $0x8506,%eax
```

因此 v1 = secret_mix(t^ u)^ r9

最后，程序将 t 右移 5 位，与 u + v0 + 0x8506 相加，再与 v1 异或，比较结果是否等于 v2：

关键汇编：

```
# 计算 t >> 5
4026d1: shr    $0x5,%esi
# edx = u + v0 + 0x8506 + (t >> 5)
4026d4: lea    0x8506(%r8,%rcx,1),%edx
4026dc: add    %esi,%edx
# v2 = v1^ edx，并与第三个输入比较
4026de: xor    %edx,%eax
4026e0: cmp    -0x24(%rbp),%eax
4026e3: sete   %dl
```

综合得到：

v0 = context & 3
t = ch_high^ ch_low
u = cx_high^ cx_low
r9 = (v0 * 0x6f35)^ cx_middle
v1 = secret_mix(t^ u)^ r9
v2 = v1^ (u + v0 + 0x8506 + (t >> 5))

构造 Honor 输入并实现：

输入需要满足两个条件，总长度在 36 到 163 字节之间且偏移 32 处开始的内容是 OPEN。因此输出格式先写 32 个空格，再写 OPEN，随后写三个十进制整数。

%32s 配合空字符串会输出 32 个空格；三个 %010u 将整数按至少 10 位、前导补零的格式输出。32 位无符号整数最多 10 位，因此每个数都占 10 位。整个字符串长度为 69 字节，满足长度限制。

最后，solve_secret 将完整的第七行写入 out。驱动程序根据 challenge 和前六关口令生成 context，并在六关口令之后提供这行 Honor 输入。

检验

非公开 seed 完整通关验证，采用 seed 6：

```
{ ./defuser --seed 6; ./defuser --secret --seed 6; } | ./bomb --seed 6
```

```
tanjiayun@localhost:~/ICS2026/ics-2026-bomblab$ { ./defuser --seed 6;
./defuser --secret --seed 6; } | ./bomb --seed 6

[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 已解除]

Congratulations! You've defused the bomb.
[SAFE] 六项检查全部通过。新存档已启用。

你们走回街上。送货司机已经发动了车：
‘昨天耽误了。今天得早点走。’
维修铺的灯装好了，老板正把那张写着‘明天装上’的纸条揭下来。
林夏站在橱窗前看了一会儿，跟上了你。

Honor · 观测台旧备份

林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。
```

使用公开测试列表：

```
make test
```

```
tanjiayun@localhost:~/ICS2026/ics-2026-bomblab$ make test
 Public tests: 60/60
 Secret seed 0: PASS
 Secret seed 1: PASS
 Secret seed 7: PASS
 Secret seed 42: PASS
 Secret seed 99: PASS
 Secret seed 123: PASS
 Secret seed 2026: PASS
 Secret seed 65535: PASS
 Secret seed 987654321: PASS
 Secret seed 18446744073709551615: PASS
 Secret public tests: 10/10
```

AI 工具使用声明：

在本次实验报告的完成过程中，我使用了 deepseek 和 workbuddy 进行排版优化和润色表达，在分析反汇编代码时，我使用 AI 辅助解释了部分复杂的寄存器传递逻辑和指令含义，帮助理解。在理解汇编信息含义后，我独立完成了代码撰写，并能保证完全理解题目和代码含义。我承诺，代码运行全部经过实操和测试。